UNIVERSITY of WASHINGTON

Python Programming & Tools for Data Science

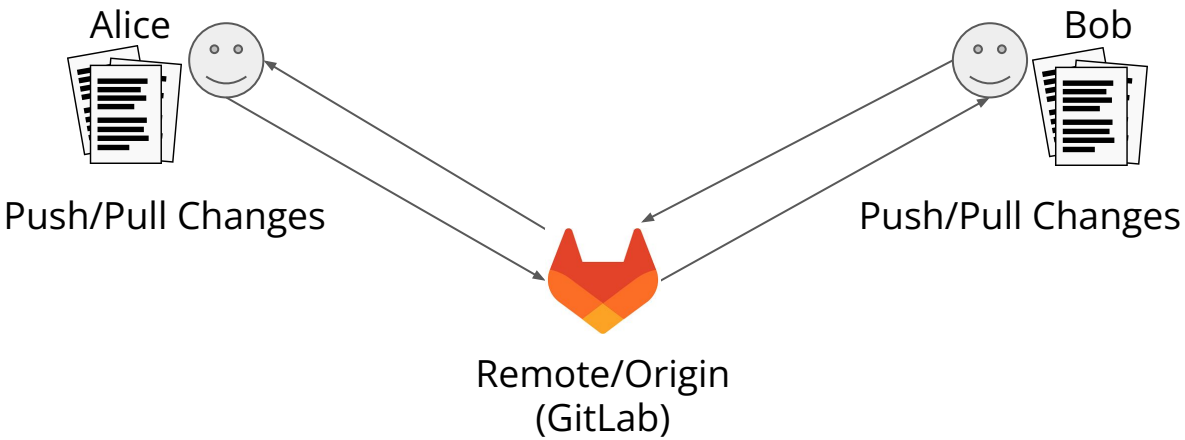
Lecture 3: Git & Jupyter Notebooks

Melissa Winstanley
University of Washington
October 15, 2026

Recap: Git

- Git is one popular, free program for **version control**
- Git helps us with **backups**, **collaboration**, and **version tracking**
- Git is good at tracking **text-based** documents
  - It *can* track non-text documents like PDFs or images, but these are less efficient

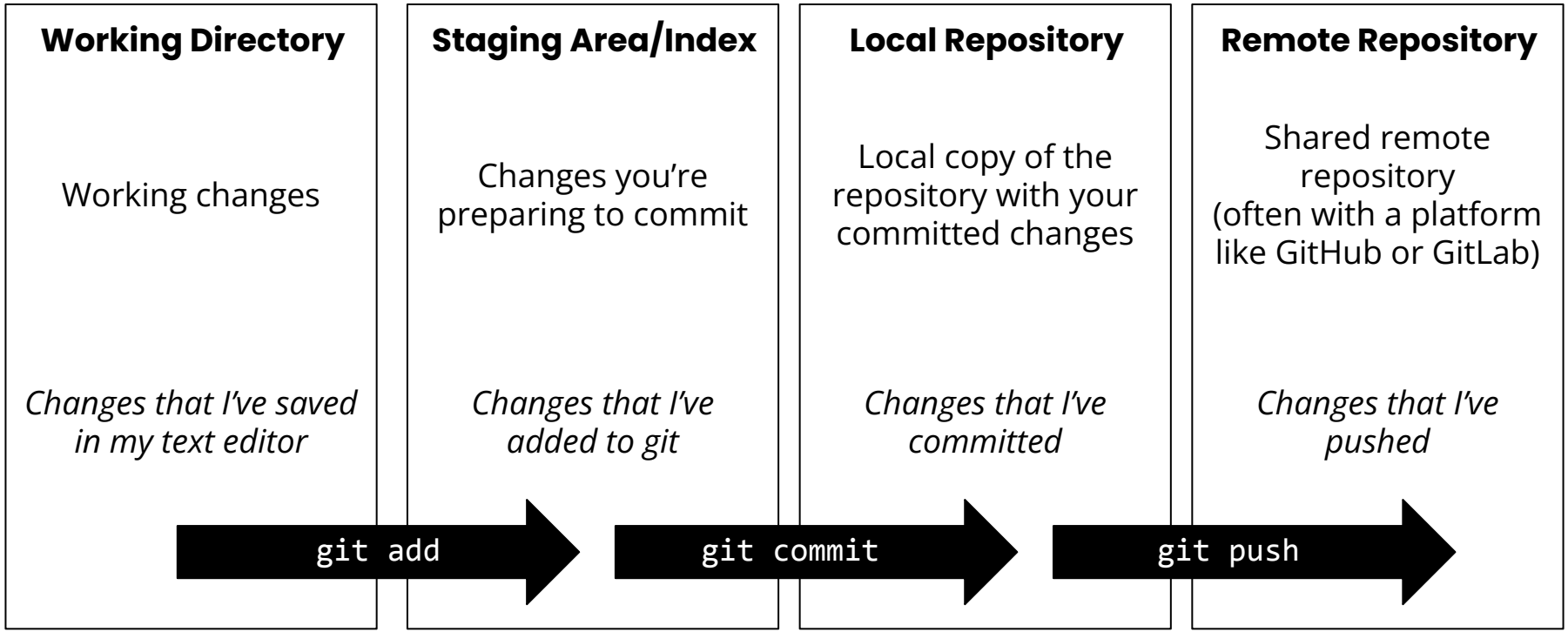
Recap: the repository model



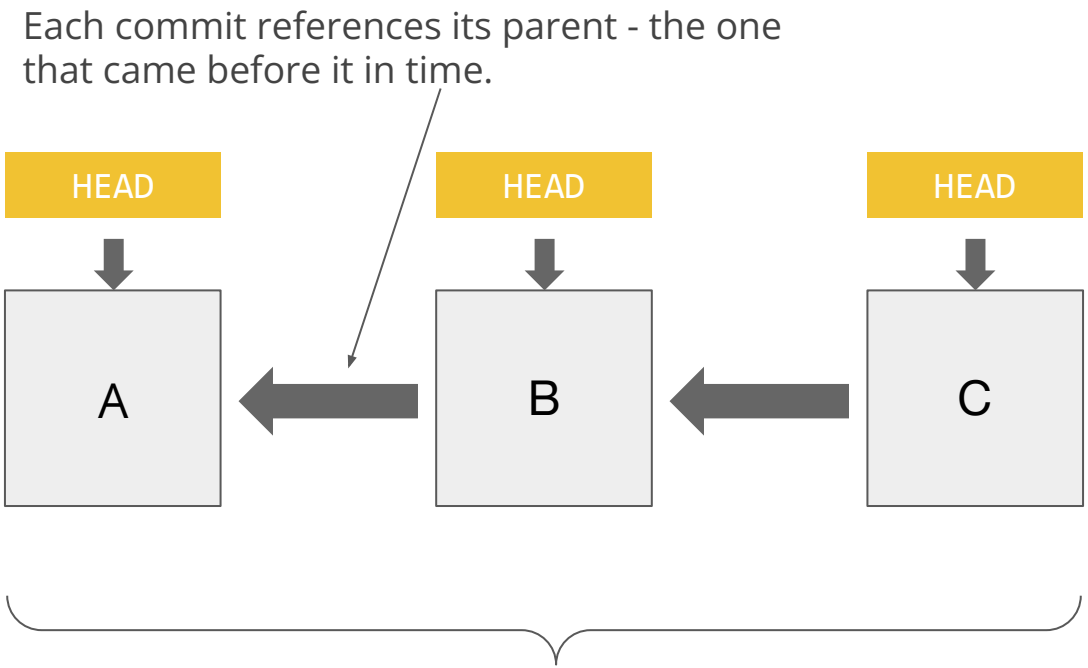
Git actions

- Clone
  - Copy the contents of a git repository onto your computer
- Add
  - Mark a file or a change to a file as “I want to keep track of this change”
- Commit
  - Save a “snapshot” of the current repository
- Push
  - Send updated commits from your computer to the remote

Git in Four Phases



Commits as a list



Observing git state

- `git status`
  - Show the current files that have been changed and/or staged for commit
- `git log`
  - Show a list of past commits
- `git diff`
  - Show unstaged changes

Demo

- GitLab
- Let’s look at a git log
- .git directory

Git branches

A **branch** in git is a pointer/reference to a specific commit.

`HEAD`: the current branch (~ commit) you are working on.

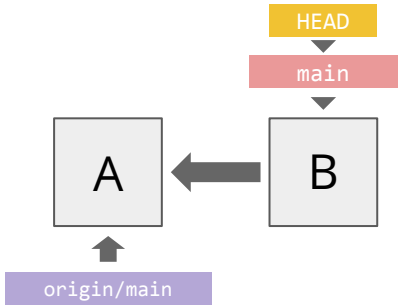
`main`: typically the “default” branch or source of truth

Why?

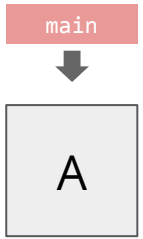
Collaboration! But we won’t need branches until DATA 515

git push (1/2)

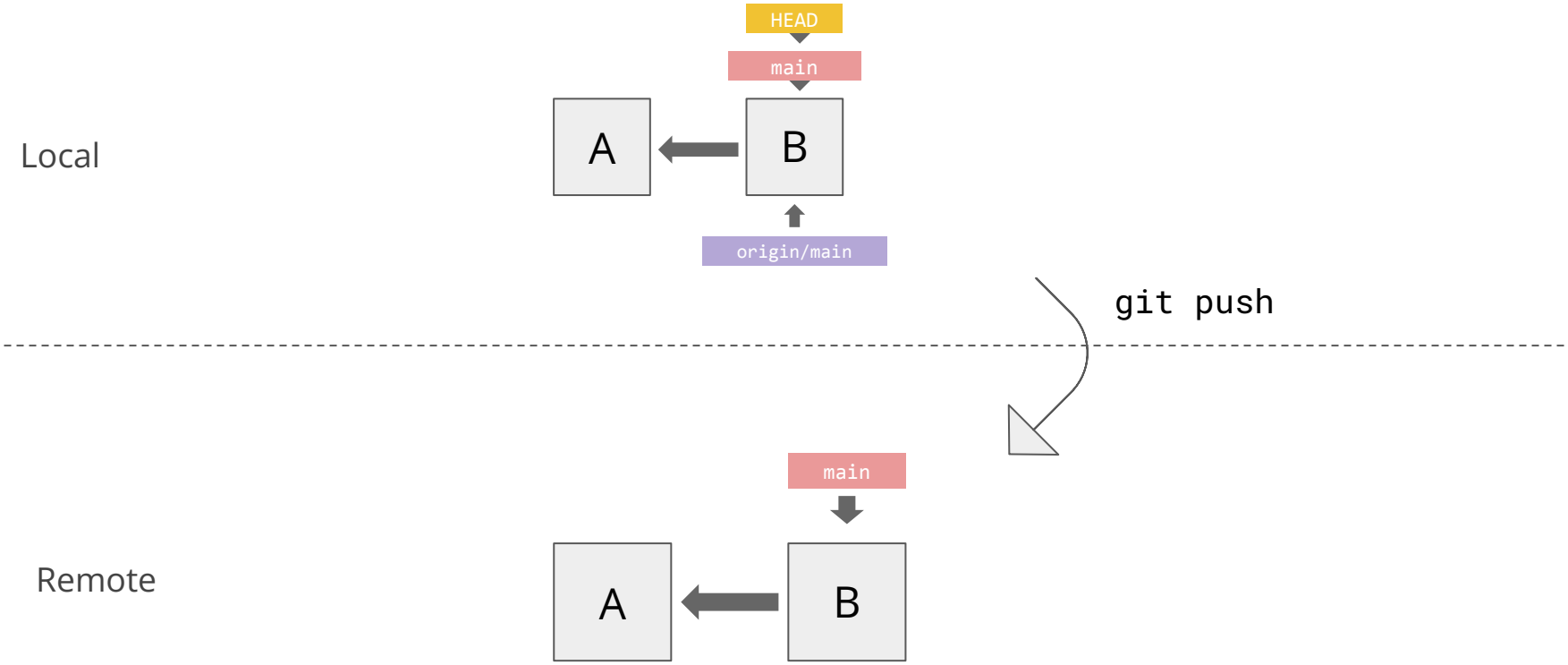
Local



Remote



git push (2/2)



Fixing problems

`git rm <file>`	remove a file
`git commit --amend`	update a commit
`git reset --hard HEAD`	throw away all working changes, reset to the most recent commit.
`git revert HEAD~1`	create a NEW commit to undo the commit 1 commit ago
`git stash`	“set aside” the current unstaged changes for later, use `git stash pop` to restore them

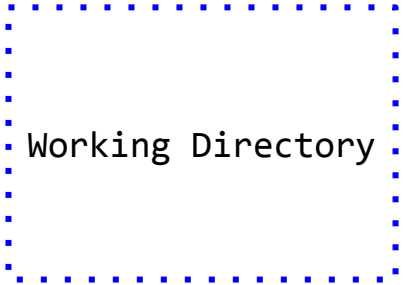
🤔 Pause and Think (1)

Suppose a run of `git status` showed the following:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
      modified:   file1.txt
      modified:   file2.txt
```

Based on this, which git phase are the changes to `file1.txt` and `file2.txt` currently in?

1. Working Directory
2. Staging Area
3. Local Repository
4. Remote Repository



🤔 Pause and Think (2)

Suppose a run of `git status` showed the following:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
      modified:   file1.txt
      modified:   file2.txt
```

Suppose we want to run a sequence of commands that:

1. includes the changes to file1.txt in a single commit
2. then, includes the changes to file2.txt in another commit
3. finally, removes `file1.txt` from the repository entirely?

What commands would we run?

```
git add file1.txt
git commit
git add file2.txt
git commit
git rm file1.txt
git commit
```

🤔 Pause and Think (3)

Suppose a run of `git status` showed the following:

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
      modified:   file1.txt
      modified:   file2.txt
```

Suppose we want to run a sequence of commands that:

1. includes the changes to file1.txt in a single commit
2. then, includes the changes to file2.txt in another commit

What commands would we run?

```
git restore --staged file2.txt
git commit
git add file2.txt
git commit
```

🤔 Pause and Think (4)

Suppose a run of `git status` showed the following:

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
      modified:   file1.txt
      modified:   file2.txt
```

Suppose we want to run a sequence of commands that:

1. throws away the changes to file2.txt (we didn't need those anyways)
2. saves the changes to file1.txt for later (we don't need it now)
3. creates and commits a new file named "Hello.txt" that contains the text "Hello World"
4. commits the changes to file1.txt

What commands would we run?

```
git restore file2.txt
git stash
echo "Hello World" > Hello.txt
git add Hello.txt
git commit
git stash pop
git add file1.txt
git commit
```

Moving on to Python